# Transit Routing System

A C++ transit routing system which uses an A* pathfinding algorithm and accelerated using Apple Metal (GPU) for parallel Haversine distance computations. It supports single and multi modal public transport networks ingested directly from GTFS datasets.

---

## Repository File Structure

```
├── a_star_router.cpp       # A* pathfinding core logic implementation
├── a_star_router.h
├── heuristic.cpp           # CPU fallback distance heuristic calculations
├── heuristic.h
├── main.cpp                # CLI entry point supporting routing and benchmarking
├── main.ipynb              # Jupyter notebook for data ingestion, cleaning, and mapping
├── metal_heuristic.h
├── metal_heuristic.mm      # Objective C++ Metal compute kernel and context manager
├── requirements.txt        # Python package dependencies
├── transit_graph.cpp       # Network graph loader and adjacency list structures
└── transit_graph.h
```

## (Note: Large datasets, compiled binaries like transit_router, and JSON outputs are excluded from the repository and are generated locally.)

## Prerequisites & Libraries

The data cleaning pipeline relies on **pandas** and **geopandas** for spatial manipulation, along with **matplotlib** and **shapely** for visualization. The router requires a C++17 compiler and Apple Metal hardware.

Install all required Python packages using:

```
pip install -r requirements.txt
```

---

## GTFS Data Setup

Download the desired GTFS schedule datasets from the Victorian Transport Open Data Portal. Place the unzipped folders into a local `./gtfs/` directory organized by their transport mode number:

- **1**: Regional Train
- **2**: Metropolitan Train
- **3**: Metropolitan Tram

- **4**: Myki Bus (Metro Bus and Regional Town Bus)
- **5**: Regional Coach
- **6**: Regional Bus
- **10**: Interstate
- **11**: SkyBus

---

## Data Preparation ( `main.ipynb` )

Before routing, prepare the raw GTFS data. Open `main.ipynb` to ingest, clean, and export your transit networks into a structured `transit_graph.json` file using the provided `DataFile` class.

### 1. Generate the Network Graph

Initialize single or multi modal feeds (e.g., combining train and tram feeds) directly in the notebook:

```
# combine metropolitan train (2) and tram (3) feeds into one unified graph
transit_data = DataFile(transport_modes=[2, 3])
transit_data.clean()
transit_data.prepare()
transit_data.export_json("transit_graph.json")
```

### 2. Finding Station IDs

Because the C++ router uses numeric Stop IDs rather than text names, search for the exact IDs interactively in Python using case insensitive queries before running terminal commands:

```
# find station IDs matching a specific name query
stations = transit_data.find_station("Central")
print(stations)
# Output displays the corresponding stop_id needed for the CLI (e.g., 10922)
```

---

## Compilation & CLI Commands

Compile the C++ router, the Objective-C++ Metal bridge, and the GPU compute shaders into a single executable:

```
clang++ -std=c++17 -O3 -I/opt/homebrew/include -framework Metal -framework Foundation -o
transit_router *.cpp *.mm
```

## Available Commands

Once compiled, execute queries via the terminal. *(Note: The Stop IDs below are randomised examples. Use the `find_station` method in your Jupyter Notebook to get valid IDs for your specific dataset).*

- **Standard CPU Route Search:** Computes the shortest path using the CPU baseline.

```
./transit_router --route 4192 8274 --time 18000
```

- **GPU Accelerated Route Search:** Offloads distance heuristic calculations to Apple Metal.

```
./transit_router --gpu --route 4192 8274 --time 18000
```

- **Dynamic Detours (Station Blocking):** Sever graph segments by blocking specific station IDs to force the algorithm to find an alternative detour.

```
./transit_router --gpu --route 4192 8274 --time 18000 --block 5931
```

- **Internal Performance Benchmarking:** Runs a 100 iteration benchmark loop directly inside the C++ binary to test pure hardware speed, bypassing Python process-spawning overhead.

```
./transit_router --benchmark --gpu --route 4192 8274 --time 18000
```